Using SQLite (sqflite) to Persist Tasks in Flutter

Overview

This document explains how the Task Management app was upgraded from **in-memory storage** to **persistent SQLite storage** using the `sqflite` package. After this change, tasks are saved to a local database on the device and survive app restarts.

What is sqflite?

`sqflite` is a Flutter plugin for SQLite — a lightweight, file-based relational database that runs entirely on the device with no server required. It is the standard choice for local persistence in Flutter apps.

Setup

Add to `pubspec.yaml`:

```yaml
dependencies:
  sqflite: ^2.4.2
  path: ^1.9.1
```

- **sqflite** — the SQLite Flutter plugin.
- **path** — provides utilities to build the database file path in a cross-platform way.

Architecture

The feature follows a **Repository pattern**:

```
UI Screens  ⟶  TasksRepository  ⟶  SQLite Database
```

All database logic lives in `TasksRepository`. Screens create an instance of it and call its methods — they never interact with the database directly. This keeps the UI code clean and makes the database layer easy to swap or test independently.

File Changes

`lib/models/task.dart` — Model Update

An `id` field was added to uniquely identify each task in the database. Two serialization methods were added to convert between Dart objects and database rows:

```dart
Map<String, Object?> toMap() {
  return {
    'id': id,
    'title': title,
    'Des': Des,
    'category': category,
    'isCompleted': isCompleted ? 1 : 0,
  };
}

static Task fromMap(Map<String, Object?> taskMap) {
  return Task(
    id: taskMap['id'] as int,
    title: taskMap['title'] as String,
    Des: taskMap['Des'] as String,
    category: taskMap['category'] as String,
    isCompleted: (taskMap['isCompleted'] as int) == 1,
  );
}
```

> SQLite has no boolean type. `isCompleted` is stored as `INTEGER` (0 = false, 1 = true) and converted back on read.

`lib/data/tasks_repository.dart` — New File

This class manages the database connection and exposes three async operations.

Opening the database:

```
Future<Database> get _database async {
  return _db ??= await openDatabase(
    join(await getDatabasesPath(), 'tasks_database.db'),
    onCreate: (db, version) {
      return db.execute(
        'CREATE TABLE tasks(id INTEGER PRIMARY KEY, title TEXT, Des TEXT, category TEXT, isCompleted INTEGER)',
      );
    },
    version: 1,
  );
}
```

The database is opened lazily (only on first use) and reused after that. The `tasks` table is created automatically on first launch via `onCreate`.

Insert:

```
Future<void> insertTask(Task task) async {
  final db = await _database;
  await db.insert('tasks', task.toMap(),
      conflictAlgorithm: ConflictAlgorithm.replace);
}
```

Read all:

```
Future<List<Task>> getTasks() async {
  final db = await _database;
  final List<Map<String, Object?>> tasksMaps = await db.query('tasks');
  return tasksMaps.map(Task.fromMap).toList();
}
```

Delete:

```
Future<void> deleteTask(int taskId) async {
  final db = await _database;
  await db.delete('tasks', where: 'id = ?', whereArgs: [taskId]);
}
```

`lib/screens/task_list_screen.dart` — Screen Update

The screen now loads tasks from the database on startup and refreshes the list after any add or delete.

```dart
@override
void initState() {
  WidgetsBinding.instance.addPostFrameCallback((_) {
    _updateTasks();
  });
  super.initState();
}

Future<void> _updateTasks() async {
  final tasks = await _taskRepository.getTasks();
  setState(() {
    _tasks = tasks;
  });
}
```

After navigating back from the Add Task screen, the list is refreshed:

```dart
onPressed: () async {
  await Navigator.push(context,
    MaterialPageRoute(builder: (context) => const AddTaskScreen()));
  _updateTasks();
},
```

`lib/screens/add_task_screen.dart` — Screen Update

On save, the task is written to the database instead of a global list. The `id` is generated from the current timestamp:

```dart
final newTask = Task(
  id: DateTime.now().microsecondsSinceEpoch,
  title: _titleController.text,
  Des: _descriptionController.text,
  category: _selectedCategory,
);

await _tasksRepository.insertTask(newTask);
```

Database Schema

Table: `tasks`

Column	Type	Notes
`id`	`INTEGER PRIMARY KEY`	Unique identifier
`title`	`TEXT`	Task title
`Des`	`TEXT`	Task description
`category`	`TEXT`	Category (Work, Personal, etc.)
`isCompleted`	`INTEGER`	0 = false, 1 = true

Before vs. After

Aspect	Before	After
Storage	Global `List<Task>` in memory	SQLite file on device
Persistence	Lost on app restart	Survives app restarts
Add task	`tasks.add(newTask)`	`repository.insertTask(task)`
Delete task	`tasks.removeAt(index)`	`repository.deleteTask(id)`
Load tasks	Hardcoded static list	Async database query
